

SMA PRACS

Dataset Link:

https://drive.google.com/file/d/1TwbH9ITJ7YfzbhYiDHkodbiYDDv-Z6lf/view?usp=drive_link

1.Content analysis: Analyze the content of social media data to determine what topics are being given data set using topic modelling, keyword extractor

LDA (Latent Dirichlet Allocation) is a **topic modeling** technique. It helps you automatically **discover hidden topics** in a large collection of documents — without needing to read them all.

TF-IDF (Term Frequency–Inverse Document Frequency) find the most repeated words from the dataset based on the frequency of each word.

```
import nltk
import numpy as np
import pandas as pd

from nltk.corpus import stopwords
from gensim import corpora, models
from gensim.utils import simple_preprocess
from sklearn.feature_extraction.text import TfidfVectorizer

data=pd.read_csv("C:/Users/Aarti
Mishra/Downloads/SMA_EXP_5_MahaKumbh_Data.csv")

data=data.drop(columns=['link'])

print(data.head())

data=data[['text']]

data1=data

print(data.head())

nltk.download('stopwords')

stop_words = stopwords.words('english')

def preprocessing(text):
    return [word for word in simple_preprocess(text) if word not in
stop_words]

data['text']=data['text'].apply(preprocessing)

print(data.head())
```

```

dictionary=corpora.Dictionary(data['text'])
corpus=[dictionary.doc2bow(doc) for doc in data['text']]
print(corpus)

lda_model=models.LdaModel(corpus=corpus,num_topics=10,id2word=dictionary,passes=10)

for idx,topic in lda_model.print_topics():
    print(f"Topic {idx} : {topic}")

data['processed_text'] = data['text'].apply(lambda x: ' '.join(x))
tfidf=TfidfVectorizer(stop_words='english',max_features=10)

X=tfidf.fit_transform(data['processed_text'])

keyword=tfidf.get_feature_names_out()

print("Topic keywords",keyword)

```

2.Location analysis: Analyze the location data associated with tweets to understand where particular location are most prevalent.

```

import pandas as pd
import matplotlib.pyplot as plt

# Load your Excel or CSV data
data=pd.read_csv("C:/Users/Aarti Mishra/Downloads/SMA_EXP_5_MahaKumbh_Data.csv")

# Check if 'Location' column exists
if 'location' not in data.columns:
    print("No 'Location' column found!")
else:
    # Count frequency of each location
    location_counts = data['location'].value_counts()

    # Print top 10 most frequent locations
    #print(location_counts.head(10))

    top_locations = location_counts.head(10)

    print(top_locations)
    # Print ranked location names without counts
    print(" Top 10 Most Prevalent Cities:\n")
    for idx, loc in enumerate(top_locations.index, start=1):
        print(f"{idx}. {loc}")

    # Optional: Visualize the top locations
    location_counts.head(10).plot(kind='bar', color='skyblue')
    plt.title("Top 10 Tweet Locations")
    plt.xlabel("Location")

```

```
plt.ylabel("Tweet Count")
plt.xticks(rotation=45)
plt.show()

location_counts.head(10).plot(kind='line', color='skyblue')
plt.title("Top 10 Tweet Locations")
plt.xlabel("Location")
plt.ylabel("Tweet Count")
plt.xticks(rotation=45)
plt.show()
```

3.Trend analysis : Analyze the time data associated with social media and analyse its trends.

```
import pandas as pd
import matplotlib.pyplot as plt

# Load your dataset
df = pd.read_csv("C:/Users/Aarti Mishra/Downloads/SMA_EXP_5_MahaKumbh_Data.csv")

# Automatically parse ISO-style datetime column
df['date'] = pd.to_datetime(df['date'], errors='coerce') # will convert correctly formatted strings

# Drop rows where parsing failed (if any)
df = df.dropna(subset=['date'])

# Calculate engagement score
df['engagement_score'] = df['likes'] + df['retweets'] + df['comments']

# Extract just the date part
df['date_only'] = df['date'].dt.date

# Group by date and sum the engagement score
daily_engagement = df.groupby('date_only')['engagement_score'].sum().reset_index()

# Plot the engagement trend
plt.figure(figsize=(10, 6))
plt.plot(daily_engagement['date_only'], daily_engagement['engagement_score'], marker='o', color='blue')
plt.title("Daily Engagement Trend - Mahakumbh 2025")
plt.xlabel("Date")
plt.ylabel("Engagement Score")
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True)
plt.show()
```

4.Hashtag popularity analysis: Determine which hashtags are most popular among different user groups.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import re

data = pd.read_csv("C:/Users/Aarti
Mishra/Downloads/SMA_EXP_5_MahaKumbh_Data.csv")

def extract_hashtags(text):
    return re.findall(r"#\w+",text)

data['hashtags']=data['text'].apply(extract_hashtags)

from collections import Counter

hashtags_flats=[tag for tags in data['hashtags'] for tag in tags]

hashtag_counts = Counter(hashtags_flats)

print(hashtag_counts)

df_hashtags=pd.DataFrame(hashtag_counts.items(),columns=['Hashtag','Count']).
sort_values(by='Count',ascending=False)
print(df_hashtags.head())

top_10=hashtag_counts.most_common(10)
print(top_10)

hashtags,counts=zip(*top_10)

plt.figure(figsize=(10,6))
plt.bar(hashtags,counts,color='red')
plt.title("Top 10 Most Popular Hashtags")
plt.xlabel("Hashtags")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

5.Sentiment Analysis for given dataset • Negative Sentiment analysis • Positive Sentiment analysis

```
import pandas as pd
import matplotlib.pyplot as plt

# Load your Excel or CSV data
data=pd.read_csv("C:/Users/Aarti
Mishra/Downloads/SMA_EXP_5_MahaKumbh_Data.csv")

# Check if 'Location' column exists
if 'location' not in data.columns:
    print("No 'Location' column found!")
else:
    # Count frequency of each location
    location_counts = data['location'].value_counts()

    # Print top 10 most frequent locations
    #print(location_counts.head(10))

    top_locations = location_counts.head(10)

    print(top_locations)
    # Print ranked location names without counts
    print(" Top 10 Most Prevalent Cities:\n")
    for idx, loc in enumerate(top_locations.index, start=1):
        print(f"{idx}. {loc}")

    # Optional: Visualize the top locations
    location_counts.head(10).plot(kind='bar', color='skyblue')
    plt.title("Top 10 Tweet Locations")
    plt.xlabel("Location")
    plt.ylabel("Tweet Count")
    plt.xticks(rotation=45)
    plt.show()

    location_counts.head(10).plot(kind='line', color='skyblue')
    plt.title("Top 10 Tweet Locations")
    plt.xlabel("Location")
    plt.ylabel("Tweet Count")
    plt.xticks(rotation=45)
    plt.show()
```

6.User engagement analysis: analyze how users engage with content on social media to understand what types of content are most engaging

```
import pandas as pd
import matplotlib.pyplot as plt
```

```

# Load data
df = pd.read_csv("C:/Users/Aarti
Mishra/Downloads/SMA_EXP_5_MahaKumbh_Data.csv")

# Fill missing values
df[['likes', 'retweets', 'comments']] = df[['likes', 'retweets',
'comments']].fillna(0)

# Calculate engagement score
df['engagement_score'] = df['likes'] + df['retweets'] + df['comments']

# Group by user and sum engagement
user_engagement =
df.groupby('username')['engagement_score'].sum().reset_index()

# Sort by highest engagement
top_users = user_engagement.sort_values(by='engagement_score',
ascending=False).head(10)

# Print top engaging users
print("Top 10 Most Engaging Users:")
print(top_users)

# Plot bar chart
plt.figure(figsize=(10, 6))
plt.bar(top_users['username'], top_users['engagement_score'], color='coral')
plt.title("Top 10 Most Engaging Users")
plt.xlabel("Username")
plt.ylabel("Total Engagement Score")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

7.Exploratory Data Analysis and visualization for given data set

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re

# Load your data
df = pd.read_csv("C:/Users/Aarti
Mishra/Downloads/SMA_EXP_5_MahaKumbh_Data.csv")

# =====
# 1. Basic Info
# =====
print("🔍 Dataset Info:")
print(df.info())
print("\n📦 Missing Values:\n", df.isnull().sum())
print("\n📊 Summary Stats:\n", df.describe())

```

```

# =====
# 2. Engagement Score
# =====
df[['likes', 'retweets', 'comments']] = df[['likes', 'retweets',
'comments']].fillna(0)
df['engagement_score'] = df['likes'] + df['retweets'] + df['comments']

# =====
# 3. Top Hashtags
# =====
def extract_hashtags(text):
    return re.findall(r"#\w+", str(text))

df['hashtags'] = df['text'].apply(extract_hashtags)
all_hashtags = [tag for sublist in df['hashtags'] for tag in sublist]
hashtags_df = pd.Series(all_hashtags).value_counts().reset_index()
hashtags_df.columns = ['Hashtag', 'Count']

# =====
# 4. Basic Sentiment (Manual Method)
# =====
positive_words = ["good", "great", "awesome", "amazing", "love", "happy",
"spiritual", "peaceful", "excited"]
negative_words = ["bad", "worst", "hate", "angry", "sad", "dirty", "crowded",
"boring"]

def classify_sentiment(text):
    if not isinstance(text, str):
        return "neutral"
    text = text.lower()
    pos = sum(word in text for word in positive_words)
    neg = sum(word in text for word in negative_words)
    if pos > neg:
        return "positive"
    elif neg > pos:
        return "negative"
    else:
        return "neutral"

df['sentiment'] = df['text'].apply(classify_sentiment)

# =====
# 5. Top Engaging Users
# =====
top_users =
df.groupby('username')['engagement_score'].sum().reset_index().sort_values(by
='engagement_score', ascending=False).head(10)

# =====
# 6. Visualizations
# =====

```

```

# Engagement distribution
plt.figure(figsize=(8, 5))
sns.histplot(df['engagement_score'], bins=30, kde=True, color='teal')
plt.title("Engagement Score Distribution")
plt.xlabel("Engagement Score")
plt.ylabel("Frequency")
plt.show()

# Sentiment pie chart
plt.figure(figsize=(6, 6))
df['sentiment'].value_counts().plot(kind='pie', autopct='%1.1f%%',
colors=['lightgreen', 'lightcoral', 'lightgrey'])
plt.title("Sentiment Distribution")
plt.ylabel("")
plt.show()

# Top hashtags
plt.figure(figsize=(10, 6))
sns.barplot(data=hashtags_df.head(10), x='Hashtag', y='Count',
palette='coolwarm')
plt.title("Top 10 Hashtags")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Top users
plt.figure(figsize=(10, 6))
sns.barplot(data=top_users, x='username', y='engagement_score',
palette='viridis')
plt.title("Top 10 Engaging Users")
plt.xticks(rotation=45)
plt.ylabel("Engagement Score")
plt.tight_layout()
plt.show()

```

8. Brand analysis: analyze the conversation around a particular brand for given dataset

```

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
from textblob import TextBlob

data= pd.read_csv("C:/Users/Aarti
Mishra/Downloads/SMA_EXP_5_MahaKumbh_Data.csv")

brand_word="kumbh"

data['text']=data['text'].astype(str)

data_brand=data[data['text'].str.contains(brand_word,case=False,na=False)]

```

```

print(f"Total tweets Mentioning '{brand_word}':", len(data_brand))

# Engagement Analysis

data_brand[['likes','retweets','comments']]=data_brand[['likes','retweets','c
omments']].fillna(0)

data_brand['enagagment_score']=data_brand['likes']+data_brand['retweets']+dat
a_brand['comments']

print(data_brand['enagagment_score'])

# Sentiment Analysis

def sentiment(text):
    return TextBlob(text).sentiment.polarity

data_brand['Sentiment']=data_brand['text'].apply(sentiment)

def label_sentiment(score):
    if score > 0:
        return "Positive"
    elif score < 0:
        return "Negative"
    else:
        return "Neutral"

data_brand['sentiment_Score']=data_brand['Sentiment'].apply(label_sentiment)

sentiment_count=data_brand['sentiment_Score'].value_counts()

print(sentiment_count)

sentiment_count.plot(kind='bar',color='blue')
plt.xlabel('Sentiment')
plt.ylabel('Count')
plt.show()

plt.figure(figsize=(8, 5))
sns.histplot(data_brand['enagagment_score'], bins=20, kde=True,
color='purple')
plt.title(f"Engagement on Tweets About '{brand_word}'")
plt.xlabel("Engagement Score")
plt.tight_layout()
plt.show()

```

9. Analyze competitor activities using social media for given problem statement

```

import pandas as pd
import matplotlib.pyplot as plt

```

```

import seaborn as sns
from textblob import TextBlob

# Load dataset
data = pd.read_csv("C:/Users/Aarti
Mishra/Downloads/SMA_EXP_5_MahaKumbh_Data.csv")

# Ensure 'text' is treated as string
data['text'] = data['text'].astype(str)

# Define competitor keywords
competitors = ["kumbh", "prayagraj", "haridwar", "nasik", "ujjain"]

# Store results
results = []

# Analyze each competitor
for brand in competitors:
    # Filter relevant tweets
    brand_data = data[data['text'].str.contains(brand, case=False,
na=False)].copy()

    # Fill missing engagement data
    brand_data[['likes', 'retweets', 'comments']] = brand_data[['likes',
'retweets', 'comments']].fillna(0)

    # Calculate engagement score
    brand_data['engagement_score'] = brand_data['likes'] +
brand_data['retweets'] + brand_data['comments']

    # Sentiment analysis
    brand_data['sentiment_score'] = brand_data['text'].apply(lambda x:
TextBlob(x).sentiment.polarity)
    brand_data['sentiment_label'] = brand_data['sentiment_score'].apply(
        lambda x: 'Positive' if x > 0 else ('Negative' if x < 0 else
'Neutral')
    )

    # Store metrics
    total_tweets = len(brand_data)
    avg_engagement = brand_data['engagement_score'].mean()
    sentiment_counts = brand_data['sentiment_label'].value_counts().to_dict()

    results.append({
        "brand": brand,
        "tweets": total_tweets,
        "avg_engagement": avg_engagement,
        "positive": sentiment_counts.get("Positive", 0),
        "neutral": sentiment_counts.get("Neutral", 0),
        "negative": sentiment_counts.get("Negative", 0)
    })

# Convert results to DataFrame

```

```

results_df = pd.DataFrame(results)
print("\nCompetitor Analysis:\n", results_df)

# -----
# 📊 Visualization
# -----

# Bar chart: Tweet Volume
plt.figure(figsize=(8, 5))
sns.barplot(x='brand', y='tweets', data=results_df, palette='Set2')
plt.title("Tweet Volume per Competitor")
plt.ylabel("Number of Tweets")
plt.xlabel("Competitor")
plt.tight_layout()
plt.show()

# Bar chart: Average Engagement
plt.figure(figsize=(8, 5))
sns.barplot(x='brand', y='avg_engagement', data=results_df,
palette='viridis')
plt.title("Average Engagement per Competitor")
plt.ylabel("Average Engagement Score")
plt.xlabel("Competitor")
plt.tight_layout()
plt.show()

# Stacked Sentiment Plot
results_df.set_index("brand")[[ "positive", "neutral", "negative"]].plot(
    kind='bar', stacked=True, figsize=(9, 6), colormap="Accent"
)
plt.title("Sentiment Distribution per Competitor")
plt.ylabel("Tweet Count")
plt.xlabel("Competitor")
plt.legend(title="Sentiment")
plt.tight_layout()
plt.show()

```

10.Social Network data analysis for community detection and influential analytics for given problem using Girvan newman algorithm / Kmean clustering algorithm

```

import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv("C:/Users/Aarti
Mishra/Downloads/SMA_EXP_5_MahaKumbh_Data.csv")

```

```
# Group by user to get total likes, retweets, comments
user_data = df.groupby("user_name")[["likes", "retweets",
"comments"]].sum().reset_index()

# Fill missing values (if any)
user_data.fillna(0, inplace=True)

# Feature set for clustering
X = user_data[["likes", "retweets", "comments"]]

# Apply KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
user_data["Cluster"] = kmeans.fit_predict(X)

# Visualize clusters (2D using likes vs retweets)
plt.figure(figsize=(8, 6))
colors = ['red', 'blue', 'green']
for i in range(3):
    cluster = user_data[user_data["Cluster"] == i]
    plt.scatter(cluster["likes"], cluster["retweets"], color=colors[i],
label=f"Cluster {i}")

plt.xlabel("Likes")
plt.ylabel("Retweets")
plt.title("User Clustering based on Engagement (KMeans)")
plt.legend()
plt.grid(True)
plt.show()

# Print top users in each cluster
for i in range(3):
    print(f"\nTop users in Cluster {i}:")
    top_users = user_data[user_data["Cluster"] == i].sort_values(by=["likes",
"retweets", "comments"], ascending=False).head(3)
    print(top_users[["user_name", "likes", "retweets", "comments"]])
```